# Promo Finder for AI Microdramas

Problem 1 · Your Name · Roll No. · Branch | Code: GitHub link | Raw outputs: Drive link | Video: link

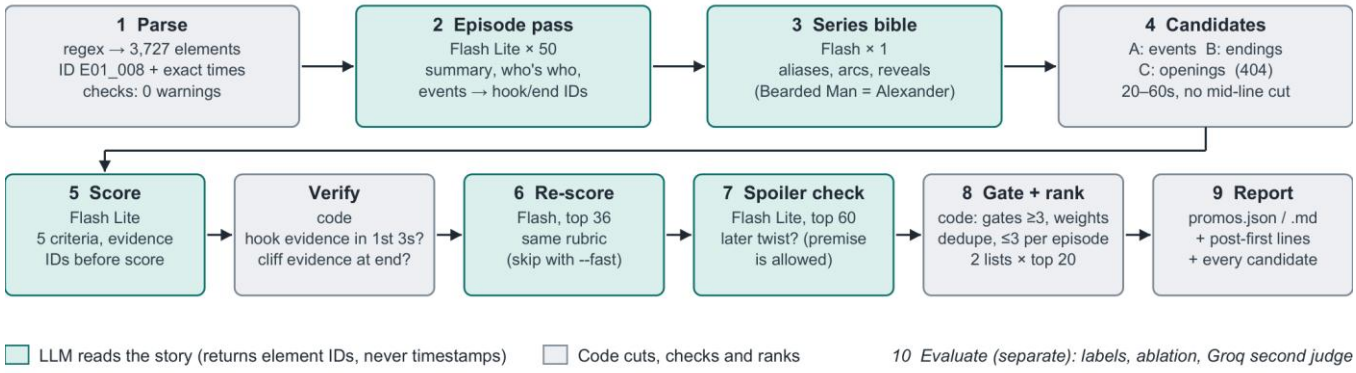
## What I built and why

Today an editor watches every episode to pick promo moments, and new shows ship every week. I built a tool that reads a show's screenplays and returns a **ranked shortlist of 20–60 s promo clips**: episode, start and end time, the hook, the cliffhanger and why it works. An editor approves the final picks. I ran it on all 50 episodes of *I Married a Broken Soldier. He Was a Duke*. I chose this problem because it is a real, repeated step at Kuku, the input is real, and the output can be measured.

## Assumptions

- **A promo is one continuous 20–60 s clip from one episode.** The brief allows stitching moments together; I left that out so every promo is easy to verify and cut.
- **The screenplay is the edit timeline.** I trust its timestamps, and treat camera and action lines as what is on screen (they describe intended shots, not the final video).
- **Hook = the first 3 seconds. Cliffhanger = the clip stops before the answer or the consequence.**
- **The viewer is a stranger scrolling past.** The show's premise (the twist in its title) is fair to show; later twists are spoilers.
- **The tool shortlists; a human decides.** It never posts anything itself.
- **Free model tiers only, and the screenplays stay private** (they are not in the public repo).

## How it works




- **AI reads the story, code makes the cuts.** The model only points to screenplay lines by ID. Code turns those into timestamps, keeps clips at 20–60 s and never cuts mid-sentence, so a promo can never point at a moment that doesn't exist.
- **Three places to look:** around key events, at each episode's ending (written as a cliffhanger), and at each episode's opening (written to grab).
- **Scores need proof.** A judge model scores hook, cliffhanger, standalone, coherence and emotion, and must quote the lines behind each score. It also sees the 8 s after the cut, to tell real suspense from a random cut. Code rejects a claim if its evidence is in the wrong place (e.g. a "hook" that isn't in the first 3 s).
- **Ranking:** a clip must score at least 3/5 on hook, cliffhanger, standalone and coherence, then is ranked by a weighted score, with at most 3 per episode. I produce two lists: one for new viewers and one to win back viewers who dropped off.
- **Easy to re-run and swap:** one command (run.py <folder>) on any show, one line in a config file to change a model, and every response is cached.

## Sample output: top 3 of 20

| # | Episode | Clip | Hook (first 3 s) | Cliffhanger |
|---|---|---|---|---|
| 1 | 15 | 0:44 – 1:26 | A thug orders Juliana to move. She refuses. | Her husband steps in; the thug challenges him to a fight. |
| 2 | 32 | 1:57 – 2:34 | The villain raises a red-hot branding iron. | Will she blind Juliana? |
| 3 | 5 | 0:56 – 1:41 | A cruel matriarch hurls Juliana's ledger into the mud. | What will the scarred husband do to her ex-fiancé? |

All three open on a threat or a humiliation, need no backstory, and cut before the consequence.

## How I checked quality

I labelled 14 episodes blind from the screenplays (AI-assisted drafts), before looking at any output for them. I used 10 to find failures and fix them (v1 → v2), and kept 4 aside to test the fixes honestly. A “match” means the clip overlaps my label by at least 50 %.

| Check | v1 | v2 |
|---|---|---|
| My promo was among the tool’s candidates (tuning episodes that have a promo) | 6 / 8 | 7 / 8 |
| My promo was in that episode’s top 3 | 5 / 8 | 7 / 8 |
| Average gap between my start/end and the tool’s | 4.9 s | 2.1 s |
| My promo was found (held-out episodes that have a promo) | 1 / 3 | 1 / 3 |
| Episodes with nothing worth posting were left empty | 3 / 3 | 3 / 3 |

- **The cliffhanger score agrees with the writers.** Endings of episodes that the next episode describes as a cliffhanger score 4.31/5, against 2.80 for other endings.
- **A second model (Groq) agrees** on hook and cliffhanger (within 0.3 points) but not on “standalone” (1.2 points apart). That is the weakest judgement in the system.
- **Honest result:** the fixes worked on the episodes I tuned on, but not yet on the 4 held-out ones. Too few to claim a general gain.

## Where it struggles

| Problem | Example | Next step |
|---|---|---|
| Misses climaxes in the middle of an episode | Ep 33: the line “That is witchcraft” (2:06) was never tried as an ending | Also try endings at the most intense lines |
| “Standalone” is hard to judge | Late-series scenes almost always fail it; the two models disagree | Calibrate with real data: 3-second hold rate, completion |
| The same idea twice | Ranks 2 and 4 (Ep 32 and Ep 31) use the same branding-iron threat | Remove near-duplicates across episodes |
| Reads text, not video | A hook may look different on screen than on paper | Check the actual frames with a vision model |

## Scale and cost

- One 50-episode show takes about 220 calls to a cheap model and 8 to a stronger one (about 1 M tokens). On free tiers that is about 15–25 minutes, because the free tier allows only 15 calls a minute. With paid keys and parallel calls it would take a few minutes. Re-runs come from the cache in about a minute.
- The stronger model only sees the top ~10 % of clips. Shows are independent, so they can run in parallel; cost grows in step with episode count.
- The editor reviews about 20 clips per show instead of watching about 100 minutes.

## AI tools

I used **Claude Code** to implement and run the pipeline, under my design decisions and review. **Gemini Flash / Flash Lite** and **Groq gpt-oss-120b** run inside the tool. **Where AI got it wrong:** the first version made short clips longer by moving their start earlier, and 131 of 204 clips silently lost their hook. I caught it by inspecting the output and changed it to extend the clip forward to the next cliffhanger instead.